Standardize Power Platform Secrets in Azure Key Vault — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High) 4

- The problem . . . 4
- The solution . . . 4
- What's delivered . . . 4
- The three biggest Commercial-vs-GCC-High differences . . . 4
- Outcome . . . 5

Solution Overview — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High) 5

- What this solution does . . . 5
- What you get . . . 5
- The deployment model (Assess → Configure → Pilot → Validate → Expand) . . . 6
- Named, reused objects . . . 6
- Commercial vs GCC High — what's different . . . 7
- Who this is for . . . 8

Requirements — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High) 8

- R1. A GCC High Power Platform tenant with a target environment . . . 8
- R2. An Azure Government subscription in the same tenant . . . 8
- R3. `Microsoft.PowerPlatform` resource provider registered . . . 9
- R4. Azure Key Vault(s) on the Azure RBAC permission model . . . 9
- R5. Key Vault Secrets User granted to the Dataverse principal and to makers . . . 9
- R6. Network path from Power Platform (US Gov) to the vault . . . 10
- R7. Admin roles and tooling . . . 10
- R8. Secret hygiene baseline . . . 10
- Requirements checklist . . . 11

Implementation Guide — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High) 11

- Phase 0 — Assess (read-only) . . . 12
- Phase 1 — Configure . . . 12
- Phase 2 — Pilot (one environment, one secret) . . . 13
- Phase 3 — Validate . . . 14
- Phase 4 — Expand . . . 15

Testing & Validation — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High) 15

- T0. Cloud/context sanity (Gate 0) . . . 16

T1. Configuration controls (Gate 1) . . . 16
T2. Pilot functional test (Gate 2) . . . 16
T3. Validation gates (Gate 3) . . . 17
T4. Expand acceptance (Gate 4 / exit) . . . 17
Evidence to capture . . . 18
Troubleshooting — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High) 18
1. "This variable didn't save properly. User isn't authorized to read secrets from 'Azure Key Vault path'." . . . 18
2. Environment variable saves, but the flow fails to retrieve the secret . . . 18
3. Everything "looks right" but nothing resolves — you're in the wrong cloud . . . 19
4. Key Vault firewall blocks Power Platform . . . 19
5. Copilot Studio agent won't use the secret (`AllowedBots` error) . . . 19
6. Secret value appears in flow run history . . . 19
7. Vault is on the vault-access-policy model / access behaves inconsistently . . . 19
8. `Register-AzResourceProvider` / `New-AzRoleAssignment` fails or hits the wrong subscription . 20
9. Cross-tenant / cross-cloud reference errors . . . 20
10. Script can't find modules or the Dataverse/Copilot principal . . . 20
Escalation checklist (capture before opening a support case) . . . 20
Best Practices — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High) 20
Vault design . . . 21
Access control (least privilege on GCC High) . . . 21
Secret hygiene . . . 21
Flow / consumer safety . . . 22
Network posture . . . 22
Operational discipline . . . 22
Deliverables Summary — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High) 22
Documents (`source/`) . . . 22
Scripts (`scripts/`) . . . 23
Packaged outputs . . . 24
Suggested engagement flow . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High)

The problem

Power Platform makers need secrets — API keys, client secrets, connection strings — to run cloud flows, custom connectors, and Copilot Studio agents. Without a standard, those secrets end up pasted in plaintext inside flow steps and configuration tables: unrotatable, unauditable, and one screenshot away from exposure. For a **government (GCC High)** organization handling ITAR/CJIS-regulated data, that is an unacceptable control gap.

The compounding risk on GCC High: the sovereign cloud is **not** the Commercial portal with a different sign-in. The admin URLs differ (`*.us`), identity runs through Microsoft Entra Government, PowerShell must target Azure Government, and some management conveniences (like PIM/JIT) are absent. Teams that follow Commercial instructions configure the wrong endpoints and hit silent failures.

The solution

Standardize on the Microsoft-supported pattern: **store every secret in Azure Key Vault and reference it from Power Platform via an environment variable of type Secret.** The value stays in Key Vault; Power Platform holds only a reference. Rotation happens once, in one place; access is governed by Azure RBAC; every read is auditable.

This **Government (GCC High) edition** delivers that pattern authored for the US sovereign cloud — correct `*.us` portals, `vault.usgovcloudapi.net` endpoints, US Gov Graph/Az connections, and an explicit **Commercial-vs-GCC-High parity map** so the tenant is configured right the first time.

What's delivered

- A phased, click-to-implement runbook (**Assess** → **Configure** → **Pilot** → **Validate** → **Expand**) using the GCC High portals, with parity caveats inline.
- **Five PowerShell scripts** — discovery, validation, governance reporting, WhatIf-by-default remediation, and a Copilot Studio scope-tag builder — all on US Gov endpoints, all **metadata-only (they never read secret values)**.
- Requirements, testing gates, troubleshooting, best practices, and a **cited GCC High parity reference**.

The three biggest Commercial-vs-GCC-High differences

1. **Different portals and endpoints.** Maker `make.high.powerapps.us`, admin `high.admin.powerplatform.microsoft.us`, Azure `portal.azure.us`, Key Vault `vault.usgovcloudapi.net`, ARM `management.usgovcloudapi.net`, sign-in `login.microsoftonline.us`. PowerShell must use `-Environment AzureUSGovernment`.
2. **The Dataverse first-party principal must be verified, not assumed.** The Commercial Dataverse app id (`00000007-…`) may not be the principal to grant in GCC High; the solution flags this "verify for GCC High" and confirms the real object before granting Key Vault Secrets User.

3. **No PIM/JIT in Azure Government.** Least privilege is achieved with scoped standing RBAC groups + Conditional Access instead of just-in-time elevation — the role design reflects this.

Outcome

Power Platform secrets are centralized, rotatable, RBAC-governed, and auditable — deployed correctly on GCC High, proven on a pilot before scale, at a fraction of the cost of a bespoke consulting engagement.

Solution Overview — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High)

What this solution does

Power Platform makers routinely need secrets — API keys, client secrets, connection strings — to authenticate cloud flows, custom connectors, and Copilot Studio agents. Left to habit, those secrets get pasted into flow steps, connection references, or configuration tables in plain text, where they are hard to rotate, impossible to audit, and easy to leak.

This solution replaces that pattern with the supported, governed approach: store every secret in **Azure Key Vault** and reference it from Power Platform using an **environment variable of type Secret**. The secret value stays in Key Vault; Power Platform holds only a *reference* (subscription, resource group, vault name, secret name). Rotation happens in one place, access is controlled by Azure RBAC, and every read is auditable.

This is the **Government (GCC High) edition**. It is authored for the US sovereign cloud, using the correct `*.us` admin portals and US Government Azure endpoints, and it calls out — explicitly — where GCC High differs from Commercial so a government tenant is configured correctly rather than with Commercial steps that silently fail.

The engagement follows the standard model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a go/no-go gate between each phase — and ships with five read-only-first PowerShell scripts (discovery, validation, reporting, safe remediation, and a Copilot Studio scope-tag builder) that connect to US Government endpoints and **never read secret values** (metadata only).

What you get

- **This overview**, including the **Commercial vs GCC High — what's different** section below.
- **Requirements guide** (`02-requirements.md`) — every prerequisite with how to verify and implement it, GCC-High-aware.
- **Implementation guide** (`03-implementation.md`) — exact ordered steps using the GCC High portals, with parity caveats inline at the step where they matter.
- **Testing guide** (`05-testing.md`) — pass/fail gates for each phase.
- **Troubleshooting guide** (`06-troubleshooting.md`) — GCC-High-specific gotchas and fixes.
- **Best practices** (`07-best-practices.md`) and a **GCC High parity reference** (`gcc-high-parity-notes.md`) with citations.

- **5 PowerShell scripts** (`scripts/`) — US Gov Graph/Az endpoints (`-Environment AzureUSGovernment`), read-only discovery/validation/report plus WhatIf-by-default remediation and scope-targeting.

The deployment model (Assess → Configure → Pilot → Validate → Expand)

Phase	Goal	Gate to advance
Assess	Inventory existing vaults, secrets (by name), permission models, and where Power Platform secrets live today. Confirm GCC High tenant, licensing, roles, and endpoint reachability.	Prerequisites met; a clean discovery report exists.
Configure	Register the `Microsoft.PowerPlatform` provider, stand up (or adopt) a vault per environment on Azure RBAC, grant Key Vault Secrets User to the Dataverse principal and makers.	Validation script reports all required controls PASS on the pilot vault.
Pilot	Create one environment-variable secret in a non-production Power Platform environment; consume it from a test flow.	Test flow retrieves the secret successfully; value never appears in run history.
Validate	Confirm access control, network posture, tagging (for Copilot Studio), and rotation behavior against the pass/fail gates.	All gates in `05-testing.md` pass.
Expand	Roll the pattern out per environment (a vault per environment), migrate remaining plaintext secrets, and hand off runbooks.	Governance report clean; operators trained.

Named, reused objects

Every object created is named `<Purpose> – <Scope> – <Stage>` and reused verbatim thereafter. Examples used throughout this solution:

- **Key vault:** `PowerPlatform Secrets – <EnvName> – Prod` (one vault per Power Platform environment).
- **Admin/maker group:** `Key Vault Secrets – Makers – <EnvName>` (Entra security group granted Key Vault Secrets User).
- **Environment variable:** `Secret – <SystemName> – <EnvName>` (type Secret, referencing the vault secret).

Commercial vs GCC High — what's different

> *This is the section that keeps a government tenant from being configured with Commercial steps. Read it before you touch anything. Fuller, cited detail is in* `gcc-high-parity-notes.md`.

1. Different portals and endpoints — not the same URL with a different login.

Purpose	Commercial	GCC High
Power Platform admin center	`admin.powerplatform.microsoft.com`	`high.admin.powerplatform.microsoft.us`
Power Apps maker portal	`make.powerapps.com`	`make.high.powerapps.us`
Power Automate	`flow.microsoft.com`	`high.flow.microsoft.us`
Azure portal	`portal.azure.com`	`portal.azure.us`
Entra sign-in	`login.microsoftonline.com`	`login.microsoftonline.us`
Key Vault data plane (DNS)	`vault.azure.net`	`vault.usgovcloudapi.net`
Azure Resource Manager	`management.azure.com`	`management.usgovcloudapi.net`

Sources: Power Apps US Government service URLs and Azure Government endpoint comparison (see `gcc-high-parity-notes.md`).

2. Identity is Microsoft Entra Government. GCC High requires the customer to use Microsoft Entra Government for identities (unlike GCC, which uses public Entra ID). Your service principals, groups, and role assignments live in the `.us` identity plane.

3. PowerShell must target the sovereign cloud. All scripts connect with `Connect-AzAccount -Environment AzureUSGovernment` (Az) and, where Graph is used, `Connect-MgGraph -Environment USGov`. Running the Commercial default silently points at `management.azure.com` and will not see your Gov resources.

4. Key Vault feature itself is at parity; the plumbing around it differs. Azure Key Vault, Azure RBAC, the **Key Vault Secrets User** role, and the `Microsoft.PowerPlatform` provider registration all exist in Azure Government. What differs is endpoints, the identity plane, network/IP allowlists (Power Platform US Gov IP ranges), and one caveat below.

5. The Dataverse first-party principal object may differ in GCC High — verify. The Commercial Dataverse app id is `00000007-0000-0000-c000-000000000000`. In GCC High the first-party service principal that must hold **Key Vault Secrets User** can surface under a different display name / object id in your tenant. The scripts flag this as **"verify for GCC High"** rather than assuming — confirm the actual principal in **Entra ID** → **Enterprise applications** before granting.

6. Some management conveniences are absent in Azure Government. Notably, **Privileged Identity Management (PIM) / eligible (JIT) role activation is not available in Azure Government** — so

least-privilege here is achieved with standing-but-scoped RBAC groups and Conditional Access, not JIT elevation. Plan role assignments accordingly.

7. Availability can lag. Microsoft targets functional parity but documents exceptions in the Business Applications US Government availability summary. Where a specific capability (for example a preview) is not confirmed for GCC High, this solution flags it **"verify for GCC High"** rather than asserting it works.

Who this is for

Government, DoD-contractor, and other ITAR/CJIS-regulated organizations running Power Platform on **GCC High** who need their makers to consume secrets the supported, auditable way — with the sovereign-cloud specifics handled.

Requirements — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High)

These solution-specific requirements are layered on top of the **Global Requirements — Azure Solutions** (`global-requirements-azure-solutions.md`, items G1–G6). Read those first; this file adds what is specific to Power Platform + Azure Key Vault secrets on a **GCC High** tenant.

Each requirement is structured **Description / Why / Verify / Implement-if-missing**. Where a point is GCC-High-specific and not confirmed in public Microsoft US Gov docs, it is flagged **"verify for GCC High."**

> *Naming convention (reused throughout): every created object is named* `<Purpose> – <Scope> – <Stage>`.

R1. A GCC High Power Platform tenant with a target environment

- **Description:** An active **Power Apps US Government — GCC High** deployment with at least one Power Platform environment (Dataverse enabled) to pilot in.
- **Why:** Environment-variable secrets are a Dataverse/solution capability; there must be a Dataverse-backed environment to hold the environment variable and the solution it lives in.
- **Verify:** Sign in to `https://high.admin.powerplatform.microsoft.us` (GCC High admin center) and confirm the environment exists with **Dataverse** provisioned. Confirm the environment region is a US Gov region (US Gov Virginia / US Gov Texas).
- **Implement-if-missing:** In the GCC High admin center, **Environments** → **New**, choose a US Gov region, and enable Dataverse. Provision through an authorized AOS-G channel if the tenant itself is not yet GCC High.

R2. An Azure Government subscription in the same tenant

- **Description:** An **Azure Government** subscription bound to the same Microsoft Entra Government tenant as the Power Platform environment.

• **Why:** The Key Vault must be in the **same tenant** as the Power Platform subscription — cross-tenant references are not supported. Government Power Platform runs on Azure Government, so the vault must be an Azure Government vault.

• **Verify:** Sign in to `https://portal.azure.us` (Azure Government portal). Confirm the subscription's directory matches the Power Platform tenant. Confirm the subscription resolves under Azure Government (ARM endpoint `management.usgovcloudapi.net`).

• **Implement-if-missing:** Obtain/associate an Azure Government subscription in the correct tenant. Do **not** use a Commercial (`portal.azure.com`) subscription — it is a different cloud and will not be reachable from GCC High Power Platform.

R3. `Microsoft.PowerPlatform` resource provider registered

• **Description:** The `Microsoft.PowerPlatform` resource provider registered on the Azure Government subscription that holds the vault.

• **Why:** Power Platform's ability to read the referenced Key Vault secret depends on this provider being registered in the subscription. (Microsoft Learn: *Use environment variables for Azure Key Vault secrets → Configure Azure Key Vault*.)

• **Verify:** `portal.azure.us` → **Subscriptions** → **Resource providers**, search `Microsoft.PowerPlatform`, confirm **Registered**. Or run `PPKVG-Discovery.ps1` / `PPKVG-Validation.ps1`.

• **Implement-if-missing:** Register it — portal (**Register**) or `Register-AzResourceProvider -ProviderNamespace Microsoft.PowerPlatform` (connected with `-Environment AzureUSGovernment`), or `PPKVG-Remediation.ps1 -Apply`.

R4. Azure Key Vault(s) on the Azure RBAC permission model

• **Description:** One Azure Government Key Vault **per Power Platform environment** (recommended), configured with **Permission model = Azure role-based access control**.

• **Why:** Microsoft's service evaluates access using the Azure RBAC APIs even if the vault is still on the legacy vault-access-policy model; standardizing on RBAC avoids ambiguity and matches current guidance. A vault per environment limits blast radius if one is breached.

• **Verify:** `portal.azure.us` → the vault → **Settings** → **Access configuration** → confirm **Azure role-based access control**. Or `PPKVG-Validation.ps1` (reports `Permission model = RBAC`).

• **Implement-if-missing:** Create the vault (`PowerPlatform Secrets – <EnvName> – Prod`) and set the RBAC model, or switch an existing vault via **Access configuration** / `PPKVG-Remediation.ps1 -Apply`.

R5. Key Vault Secrets User granted to the Dataverse principal and to makers

• **Description:** The **Key Vault Secrets User** role assigned at the vault scope to (a) the **Dataverse first-party service principal** and (b) the makers/operators who create or use the environment variables (via a named Entra group).

• **Why:** Dataverse retrieves the secret on the maker's behalf, so **both** the Dataverse principal and the user need read access to secrets; without it the environment variable fails to save/resolve.

• **Verify:** `portal.azure.us` → vault → **Access control (IAM)** → **Role assignments**, confirm **Key Vault Secrets User** for the makers group and the Dataverse app. Or `PPKVG-Validation.ps1`.

• **Implement-if-missing:** Assign the role (IAM → Add role assignment) or run `PPKVG-Remediation.ps1 -Apply`.

• **GCC High caveat — verify for GCC High:** The Commercial Dataverse app id is `00000007-0000-0000-c000-000000000000`. In GCC High the first-party principal may appear under a different display name/object id. **Confirm the actual principal** in **Entra ID** → **Enterprise applications** before assigning; do not assume the Commercial id resolves.

R6. Network path from Power Platform (US Gov) to the vault

• **Description:** The vault reachable by Power Platform — either public network access with the **Power Platform US Government IP ranges** allowed through the Key Vault firewall, or a **Virtual Network / Private Link** path delegated to Power Platform.

• **Why:** If the Key Vault firewall is on, Power Platform is **not** covered by the "Trusted Services" allowance and must be explicitly allowed; otherwise secret retrieval is blocked.

• **Verify:** vault → **Networking**. If firewall enabled, confirm the current Power Platform US Gov IP ranges are allowed (see Power Platform URLs and IP address ranges for US Gov). Or confirm a Private Link path to a Power Platform-delegated VNet.

• **Implement-if-missing:** Allow the Power Platform US Gov IP ranges, or configure VNet support / Private Link per `03-implementation.md`. **Verify for GCC High:** use the US Gov IP range set, not the Commercial one.

R7. Admin roles and tooling

• **Description: Owner** or **User Access Administrator** on the vault's subscription/resource group (to grant RBAC), plus **System Administrator** in the target Power Platform environment (to create solutions/environment variables). PowerShell 5.1+ or 7+ with **Az.Accounts, Az.KeyVault, Az.Resources**; for any Graph steps, **Microsoft.Graph**.

• **Why:** Role assignment requires an access-admin role; creating the environment variable requires Power Platform admin/maker rights; the scripts require the Az modules and US Gov connection.

• **Verify:** Confirm roles in `portal.azure.us` (IAM) and the GCC High admin center. Confirm modules: `Get-Module Az.KeyVault -ListAvailable`.

• **Implement-if-missing:** Assign the roles (prefer a scoped group over standing individual grants — **note: PIM/JIT is not available in Azure Government**, so use scoped standing access + Conditional Access). Install modules: `Install-Module Az -Scope CurrentUser`.

R8. Secret hygiene baseline

• **Description:** A naming and rotation standard for secrets (e.g. `Secret – <SystemName> – <EnvName>`), an expiry policy, and, for Copilot Studio consumers, the `AllowedBots` / `AllowedEnvironments` tag plan.

• **Why:** Standardized names and expiry make rotation and audit repeatable; Copilot Studio will not consume a secret unless the correct scope tag is present.

• **Verify:** Run `PPKVG-Report.ps1` — it flags secrets with no expiry and shows current tags (names/metadata only).

• **Implement-if-missing:** Adopt the naming/expiry standard; set scope tags with `PPKVG-Set-ScopeTargeting.ps1` (WhatIf first).

Requirements checklist

#	Requirement	Verify with
R1	GCC High Power Platform tenant + Dataverse environment	`high.admin.powerplatform.microsoft.us`
R2	Azure Government subscription, same tenant	`portal.azure.us`
R3	`Microsoft.PowerPlatform` provider registered	`PPKVG-Validation.ps1`
R4	Key Vault(s) on Azure RBAC model	`PPKVG-Validation.ps1`
R5	Key Vault Secrets User → Dataverse + makers	`PPKVG-Validation.ps1` (verify Dataverse principal for GCC High)
R6	Network path Power Platform → vault (US Gov IPs / Private Link)	vault → Networking
R7	Admin roles + Az modules (no PIM in Gov)	IAM / `Get-Module`
R8	Secret naming, expiry, Copilot scope tags	`PPKVG-Report.ps1`

Implementation Guide — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High)

This is the ordered, click-to-implement procedure for a novice admin, using the **GCC High** portals throughout. GCC-High-only caveats appear inline at the step where they matter. Follow the phases in order and honor each gate.

> ***Sovereign portals used here:*** *Azure Government portal —* `https://portal.azure.us` *Power Platform admin center (GCC High) —* `https://high.admin.powerplatform.microsoft.us` *Power Apps maker (GCC High) —* `https://make.high.powerapps.us` *Copilot Studio —* `https://copilotstudio.microsoft.com` *(sign in with your GCC High identity) Entra sign-in endpoint —* `https://login.microsoftonline.us`
>
> ***Objects you will create and reuse (names):*** *Vault* `PowerPlatform Secrets – <EnvName> – Prod` · *Group* `Key Vault Secrets – Makers – <EnvName>` · *Environment variable* `Secret –`

`<SystemName> – <EnvName>`.

Phase 0 — Assess (read-only)

0.1 Confirm you are in the right cloud. Open `https://portal.azure.us` and sign in. In the top-right account pill, confirm the directory is your **GCC High / Entra Government** tenant. If your bookmark went to `portal.azure.com`, stop — that is Commercial and will not show your Gov resources.

0.2 Run discovery. From an admin workstation:

```
powershell -ExecutionPolicy Bypass -File .\PPKVG-Discovery.ps1
# prompts a Gov sign-in (Connect-AzAccount -Environment AzureUSGovernment)
```

Review the CSV: existing vaults, their permission model, secret **names** (never values), and Key Vault Secrets User assignments. Note where Power Platform secrets currently live in plaintext (flow steps, connection references, config tables) — these are your migration backlog.

Gate 0 → **1:** Requirements R1–R2 and R7 met; a clean discovery report exists.

Phase 1 — Configure

1.1 Register the `Microsoft.PowerPlatform` resource provider

1. `https://portal.azure.us` → search **Subscriptions** → select the subscription that will hold the vault.
2. Left nav → **Settings** → **Resource providers**.
3. Search `Microsoft.PowerPlatform`. If **Status** is not **Registered**, select it → **Register**. Wait until it shows **Registered** (refresh).

(Or run `PPKVG-Remediation.ps1` in preview, then `-Apply`.)

1.2 Create the Key Vault (one per environment)

1. `https://portal.azure.us` → **Create a resource** → search **Key Vault** → **Create**.
2. **Basics:** choose the subscription and resource group; **Name** = `PowerPlatform Secrets – <EnvName> – Prod` (or your naming standard — vault names have character limits, so use a compliant short form and record the mapping); **Region** = a **US Gov** region (US Gov Virginia or US Gov Texas). **Do not** include ITAR/export-controlled data in the resource name.
3. **Access configuration** tab → **Permission model** → select **Azure role-based access control**. *(This is the recommended model; Power Platform evaluates access via RBAC regardless.)*
4. **Networking** tab → for the pilot you may leave **Public access** enabled; you will lock this down in 1.5. **Review + create** → **Create.**
5. Confirm the vault's data-plane DNS is the sovereign suffix — the vault URI ends in `.vault.usgovcloudapi.net` (not `.vault.azure.net`). If you see `.vault.azure.net`, you created it in Commercial by mistake.

1.3 Create the makers group and grant Key Vault Secrets User

1. Create (or reuse) an Entra Government security group `Key Vault Secrets – Makers – <EnvName>` and add the makers/operators who will create or use the environment variable.

2. In the vault → **Access control (IAM)** → **Add** → **Add role assignment**.

3. **Role** tab → select **Key Vault Secrets User** → **Next**.

4. **Members** → **Select members** → pick the makers group → **Review + assign**.

1.4 Grant Key Vault Secrets User to the Dataverse principal

> ***GCC High caveat — verify for GCC High.*** *In Commercial the Dataverse first-party app is `00000007-0000-0000-c000-000000000000`. In GCC High the first-party principal that Dataverse uses may appear under a different display name/object id.* ***Confirm the actual principal first.***

1. In a new tab open **Entra ID** → **Enterprise applications** (via `portal.azure.us`). Search for the Dataverse / Power Platform first-party application and note its **Object ID**. If you cannot positively identify it, treat this as **"verify for GCC High"** and confirm with Microsoft support / your AOS-G partner before proceeding.

2. Back in the vault → **Access control (IAM)** → **Add** → **Add role assignment**.

3. **Role** = **Key Vault Secrets User** → **Next**.

4. **Members** → **Assign access to = User, group, or service principal** → **Select members** → choose the confirmed Dataverse principal → **Review + assign**.

5. If multiple Dataverse service principals exist in the tenant, assign them all, then review each and remove any whose Application ID is not the legitimate Dataverse identity you confirmed in step 1.

(`PPKVG-Remediation.ps1` automates 1.1, 1.2 RBAC switch, and 1.3/1.4 — but it will REVIEW-flag the Dataverse principal if it cannot resolve the id, exactly because of this caveat.)

1.5 Lock down the vault network (US Gov IP ranges)

If you will enforce the Key Vault firewall:

1. Vault → **Networking** → **Firewalls and virtual networks** → **Enabled from selected networks**.

2. Under **Firewall**, add the **Power Platform US Government IP ranges** (from *Power Platform URLs and IP address ranges* — use the **US Gov** set, focusing on `AzureCloud.usgovvirginia` / `AzureCloud.usgovtexas`). **Power Platform is not covered by "Allow trusted Microsoft services,"** so you must add its IPs explicitly.

3. For a no-public-internet posture, instead configure **VNet support for Power Platform + Private Link**: disable public access, create a private endpoint on the Power-Platform-delegated subnet, and validate connectivity. (See Microsoft: *Set up Virtual Network support for Power Platform*.)

Gate 1 → 2: Run `PPKVG-Validation.ps1` against the pilot vault — every required control **PASS** (provider registered, RBAC model, Key Vault Secrets User present, network reachable).

Phase 2 — Pilot (one environment, one secret)

2.1 Add the secret to the vault (value stays in Key Vault)

1. `https://portal.azure.us` → the vault → **Objects** → **Secrets** → **+ Generate/Import**.

2. **Name** = `Secret – <SystemName> – <EnvName>`; **Value** = the secret; optionally set an **Expiration date** (recommended). **Create.**

The scripts in this solution never read this value. Only a human, in the portal, sets it.

2.2 Create the environment-variable secret in Power Platform

1. `https://make.high.powerapps.us` → confirm the correct **environment** (top-right environment picker) = your pilot environment.

2. **Solutions** → open (or create) an **unmanaged** solution for development.

3. **New** → **More** → **Environment variable**.

4. **Display name** = `Secret – <SystemName> – <EnvName>`; add a description.

5. **Data type = Secret**; **Secret Store = Azure Key Vault**.

6. Select **New Azure Key Vault value reference** and enter:

• **Azure Subscription ID** — the Gov subscription id (vault **Overview** page).

• **Resource Group Name** — the vault's resource group.

• **Azure Key Vault Name** — the vault name.

• **Secret Name** — `Secret – <SystemName> – <EnvName>`.

1. **Save.**

*If you get *"User isn't authorized to read secrets from 'Azure Key Vault path'"*, the signed-in maker is missing **Key Vault Secrets User** on that vault (R5). Fix the role, then re-save.*

2.3 Consume the secret from a test cloud flow

1. In the same solution → **New** → **Automation** → **Cloud flow** → **Instant** → trigger **Manually trigger a flow** → **Create**.

2. **New step** → **Microsoft Dataverse** → **Perform an unbound action**.

3. **Action name** = **RetrieveEnvironmentVariableSecretValue**. Provide the environment variable's **unique (schema) name** (not the display name).

4. Rename the step (e.g. **GetSecret**). Open **…** → **Settings** and enable **Secure Outputs** so the value never lands in run history.

5. Add an **HTTP** step (or your real consumer) that uses the secret — bind the value from `body('GetSecret')?['EnvironmentVariableSecretValue']`. On the HTTP step, **…** → **Settings**, enable **Secure Inputs** and **Secure Outputs**.

6. **Save**, then **Run** manually.

Gate 2 → 3: The flow runs successfully and the secret resolves; the value does **not** appear anywhere in run history (secure in/out confirmed).

Phase 3 — Validate

1. **Access control:** `PPKVG-Validation.ps1` → all required controls PASS; Dataverse principal confirmed for GCC High.

2. **Network:** with the firewall enabled (1.5), the pilot flow still resolves the secret (proves the US Gov IP allowance / Private Link works).

3. **Rotation:** in the portal, add a **new version** of the secret; confirm the flow picks up the new value without editing the environment variable (reference is version-agnostic to current).

4. **Governance report:** `PPKVG-Report.ps1` → review Findings (no missing-expiry, no access-policy vaults).

3a. Copilot Studio consumer (only if in scope)

Copilot Studio needs two extra things beyond a flow:

1. Grant **Key Vault Secrets User** to the **Microsoft Copilot Studio Service** principal on the vault (IAM → Add role assignment) — **verify for GCC High** that this principal resolves in your Gov tenant.

2. Scope the secret with a **tag**: either `AllowedBots = {envId}/{schemaName}` (specific agents) or `AllowedEnvironments = {environmentId}` (all agents in an environment). Build/apply the tag with:

```
# preview first
powershell -ExecutionPolicy Bypass -File .\PPKVG-Set-ScopeTargeting.ps1 -AllowedEnvironments 'your-environ
# then apply
powershell -ExecutionPolicy Bypass -File .\PPKVG-Set-ScopeTargeting.ps1 -AllowedEnvironments 'your-environ
```

Test the agent topic; if it errors with an `AllowedBots` message, the error text gives you the exact `{envId}/{schemaName}` value to tag.

Gate 3 → **4:** All pass/fail gates in `05-testing.md` pass.

Phase 4 — Expand

1. Repeat Phase 1–3 **per Power Platform environment** — one vault each (`PowerPlatform Secrets – <EnvName> – Prod`).

2. Work the migration backlog from Phase 0: for each plaintext secret, create the vault secret + environment variable, repoint the consumer, then remove the plaintext copy.

3. Standardize expiry/rotation and run `PPKVG-Report.ps1` on a schedule; hand off the runbooks in `ADMIN_DOCS` and the troubleshooting guide.

Exit: Governance report clean across environments; operators trained; no plaintext secrets remain in flows/connectors.

Testing & Validation — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High)

Each test is **pass/fail** and maps to a phase gate in `03-implementation.md`. Run them in order. All PowerShell tests connect to the sovereign cloud (`-Environment AzureUSGovernment`) and read **metadata only — never secret values**.

> *Portals: Azure Government* `https://portal.azure.us` · *Power Apps (GCC High)* `https://make.high.powerapps.us` · *Power Platform admin (GCC High)* `https://high.admin.powerplatform.microsoft.us`.

T0. Cloud/context sanity (Gate 0)

Step	Expected (PASS)	FAIL means
Sign in to `portal.azure.us`; check directory	Directory = your GCC High / Entra Government tenant	You are in Commercial (`portal.azure.com`) — wrong cloud
`PPKVG-Discovery.ps1` runs and prints ARM endpoint	ARM = `management.usgovcloudapi.net`	Endpoint is `management.azure.com` → not connected to Gov
Vault URIs in the report	End in `.vault.usgovcloudapi.net`	`.vault.azure.net` → vault is in Commercial

T1. Configuration controls (Gate 1)

Run: `powershell -ExecutionPolicy Bypass -File .\PPKVG-Validation.ps1 -VaultName 'your-vault-name' -SecretName 'your-secret-name'`

Control	PASS	FAIL
Sovereign endpoint	ARM on `usgovcloudapi.net`	Commercial endpoint
`Microsoft.PowerPlatform` provider	Registered	NotRegistered → run remediation
Key Vault exists	Found in subscription	Not found / wrong subscription
Permission model	Azure RBAC	Vault access policy (REVIEW → switch to RBAC)
Key Vault Secrets User	≥1 assignment at vault scope	None → Dataverse + makers cannot read
Dataverse principal	Present (confirm object id)	REVIEW — **verify for GCC High** in Enterprise applications
Network access	Reachable (public allowed, or IPs/Private Link configured)	Firewall blocks Power Platform
Target secret	Secret name exists (value not read)	Secret missing

Gate 1 PASS = overall PASS with the Dataverse principal manually confirmed for GCC High.

T2. Pilot functional test (Gate 2)

Step	Expected (PASS)	FAIL means
Save the environment-variable secret in `make.high.powerapps.us`	Saves with no authorization error	Maker lacks Key Vault Secrets User (R5)

Step	Expected (PASS)	FAIL means
Run the test cloud flow with **RetrieveEnvironmentVariableSecretValue**	Flow succeeds; secret resolves	Provider/role/network/secret-name issue (see T1)
Inspect flow **run history**	Secret value is **masked/absent** (Secure Outputs on)	Value visible → Secure In/Out not enabled — remediate immediately

T3. Validation gates (Gate 3)

Test	How	PASS
Access enforced	Sign in as a user **not** in the makers group and try to save an env-var secret	Denied with the "isn't authorized" error
Network posture	Enable the vault firewall with US Gov IP allowances (or Private Link), re-run the flow	Flow still resolves the secret
Rotation	Add a **new version** of the secret in the portal; re-run the flow	New value used with no env-var edit
Governance hygiene	`PPKVG-Report.ps1`	Findings section empty (RBAC everywhere, provider registered, secrets have expiry)

T3a. Copilot Studio (only if in scope)

Test	PASS
Copilot Studio Service principal has Key Vault Secrets User (**verify for GCC High**)	Role present
Scope tag set via `PPKVG-Set-ScopeTargeting.ps1 -Apply`	`AllowedBots` or `AllowedEnvironments` tag on the secret
Agent topic resolves the secret in the test pane	Value returned; no `AllowedBots` error

T4. Expand acceptance (Gate 4 / exit)

Test	PASS
Each environment has its own vault on RBAC	`PPKVG-Report.ps1` shows one vault per env, all RBAC
Migration backlog cleared	No plaintext secrets remain in flows/connectors/config tables
Operators trained	Runbooks in `ADMIN_DOCS` acknowledged; troubleshooting guide reviewed

Evidence to capture

• `PPKVG-Discovery` CSV, `PPKVG-Validation` console output (overall PASS), and `PPKVG-Report` HTML per environment.

• A screenshot of a flow run history showing the secret **masked** (proves Secure Outputs).

• The confirmed **Dataverse principal object id** used for the Key Vault Secrets User grant (the GCC High verification artifact).

*None of the evidence should ever contain a secret **value**. If a value appears anywhere, treat it as an incident: rotate the secret and fix Secure Inputs/Outputs.*

Troubleshooting — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High)

Symptoms, causes, and fixes — with the GCC-High-specific gotchas called out. Work top to bottom; most failures are one of the first four.

Portals: Azure Government `https://portal.azure.us` *· Power Apps (GCC High)* `https://make.high.powerapps.us` *· Power Platform admin (GCC High)* `https://high.admin.powerplatform.microsoft.us`*.*

1. "This variable didn't save properly. User isn't authorized to read secrets from 'Azure Key Vault path'."

• **Cause:** the signed-in maker lacks **Key Vault Secrets User** on that vault (access is validated in the background when you save).

• **Fix:** add the maker (or their group `Key Vault Secrets – Makers – <EnvName>`) to **Key Vault Secrets User** on the vault (IAM → Add role assignment), then re-save. Verify with `PPKVG-Validation.ps1`.

2. Environment variable saves, but the flow fails to retrieve the secret

• **Cause A — Dataverse principal missing the role.** Dataverse reads the secret on the maker's behalf; it also needs **Key Vault Secrets User**.

• **Fix:** grant it (implementation §1.4). **GCC High gotcha (verify for GCC High):** the Commercial Dataverse app id `00000007-0000-0000-c000-000000000000` may not be the principal in your GCC High tenant — confirm the real one in **Entra ID** → **Enterprise applications** before granting. Do not assume the Commercial id resolves.

• **Cause B — provider not registered.** `Microsoft.PowerPlatform` must be **Registered** on the vault's subscription.

• **Fix:** register it (Subscriptions → Resource providers) or `PPKVG-Remediation.ps1 -Apply`.

• **Cause C — network block** (see §4).

3. Everything "looks right" but nothing resolves — you're in the wrong cloud

• **GCC High gotcha.** A Commercial bookmark (`portal.azure.com`, `make.powerapps.com`) or a PowerShell session without `-Environment AzureUSGovernment` operates in a **different cloud** and cannot see your Gov resources.

• **Tells:** vault URI ends in `.vault.azure.net` (should be `.vault.usgovcloudapi.net`); script ARM endpoint is `management.azure.com` (should be `management.usgovcloudapi.net`); sign-in went through `login.microsoftonline.com` not `login.microsoftonline.us`.

• **Fix:** use the `.us` portals (§ header) and connect scripts with `Connect-AzAccount -Environment AzureUSGovernment` (and `Connect-MgGraph -Environment USGov` for Graph). `PPKVG-Validation.ps1` will REVIEW-flag a non-sovereign endpoint.

4. Key Vault firewall blocks Power Platform

• **Cause:** the vault firewall is enabled and Power Platform's IPs are not allowed. **Power Platform is NOT covered by "Allow trusted Microsoft services."**

• **GCC High gotcha:** you must allow the **US Gov** Power Platform IP ranges (focus on `AzureCloud.usgovvirginia` / `AzureCloud.usgovtexas`), not the Commercial set.

• **Fix:** vault → **Networking** → add the current Power Platform US Gov IP ranges (from *Power Platform URLs and IP address ranges*, US Gov), or move to **VNet + Private Link** delegated to Power Platform. Re-test the flow.

5. Copilot Studio agent won't use the secret (`AllowedBots` error)

• **Cause:** Copilot Studio needs (a) **Key Vault Secrets User** for the **Microsoft Copilot Studio Service** principal, and (b) a scope **tag** on the secret.

• **Fix:** grant the role (verify for GCC High that the principal resolves), then set the tag. The error text gives the exact `{envId}/{schemaName}` — apply it with `PPKVG-Set-ScopeTargeting.ps1 -AllowedBots '<envId>/<schemaName>' -Apply`, or scope the whole environment with `-AllowedEnvironments '<environmentId>'`. Refresh the test pane.

6. Secret value appears in flow run history

• **Cause: Secure Inputs/Outputs** not enabled on the retrieve step and/or the consuming step.

• **Fix:** on the Dataverse `RetrieveEnvironmentVariableSecretValue` step and any step using the value → **…** → **Settings** → enable **Secure Inputs** and **Secure Outputs**. **Then treat the exposure as an incident: rotate the secret** (new version in Key Vault).

7. Vault is on the vault-access-policy model / access behaves inconsistently

• **Cause:** the service evaluates access via the **Azure RBAC** APIs even when the vault still uses vault access policies, so a policy-only grant can look present but not satisfy the check.

• **Fix:** vault → **Settings** → **Access configuration** → switch **Permission model** to **Azure role-based access control**, then (re)assign **Key Vault Secrets User**. `PPKVG-Remediation.ps1 -Apply` does the switch.

8. `Register-AzResourceProvider` / `New-AzRoleAssignment` fails or hits the wrong subscription

• **Cause:** not connected to Azure Government, wrong subscription context, or insufficient rights (need **Owner**/**User Access Administrator** to assign RBAC).

• **Fix:** `Connect-AzAccount -Environment AzureUSGovernment`; `Set-AzContext -Subscription 'your-subscription-id'`; confirm your role in IAM. Re-run in preview before `-Apply`.

9. Cross-tenant / cross-cloud reference errors

• **Cause:** the Key Vault and the Power Platform environment are not in the **same tenant**, or the vault is a Commercial vault while Power Platform is GCC High.

• **Fix:** the vault must be an **Azure Government** vault in the **same Entra Government tenant** as the environment. Recreate the vault in the correct subscription/tenant if needed.

10. Script can't find modules or the Dataverse/Copilot principal

• **Modules:** `Install-Module Az -Scope CurrentUser` (needs Az.Accounts, Az.KeyVault, Az.Resources); `Install-Module Microsoft.Graph` if using Graph steps.

• **Principal not resolvable:** this is expected in GCC High and is **why the scripts REVIEW-flag it** — confirm the Dataverse and Copilot Studio Service principals manually in **Entra ID** → **Enterprise applications** and grant Key Vault Secrets User to the confirmed object. Flagged **verify for GCC High** throughout.

Escalation checklist (capture before opening a support case)

1. `PPKVG-Validation.ps1` output (overall result + the failing control).
2. `PPKVG-Report.ps1` HTML (vaults, models, Secrets User assignments — no secret values).
3. The confirmed **Dataverse principal object id** you granted (the GCC High verification artifact).
4. Which cloud/endpoints you used (must be the `.us` / `usgovcloudapi.net` set).
5. A flow run history screenshot showing the secret **masked** (never include an exposed value in a ticket).

Best Practices — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High)

Durable practices for running this pattern well on a **GCC High** tenant. GCC-High-specific guidance is called out; where a point is not confirmed in public US Gov docs it is flagged **"verify for GCC High."**

Vault design

- **One vault per Power Platform environment.** Isolate blast radius; a breach or misconfiguration of one environment's vault does not expose another's secrets. Name each `PowerPlatform Secrets – <EnvName> – Prod` and reuse the name verbatim.
- **Use the Azure RBAC permission model, not vault access policies.** Power Platform evaluates access through the Azure RBAC APIs regardless, so RBAC is the unambiguous, current-guidance choice. Switch legacy vaults over.
- **Keep the vault in the same tenant and the same sovereign cloud** as the Power Platform environment. Cross-tenant references are unsupported, and a Commercial (`vault.azure.net`) vault is unreachable from GCC High Power Platform — always use `vault.usgovcloudapi.net`.
- **Never put export-controlled or sensitive data in resource names.** Azure Government resource names are not the place for ITAR/CUI markings.

Access control (least privilege on GCC High)

- **Grant Key Vault Secrets User to a named group**, not to individuals — `Key Vault Secrets – Makers – <EnvName>`. Manage membership, not dozens of role assignments.
- **Grant only Key Vault Secrets User** to Dataverse and makers — not Contributor or Key Vault Administrator. Secrets User is read-of-secret-value only; management stays separate.
- **PIM/JIT is not available in Azure Government.** You cannot make these roles "eligible" with just-in-time activation. Achieve least privilege with **tightly scoped standing RBAC groups** + **Conditional Access** (named locations, device/compliance) rather than JIT elevation. Review membership on a schedule.
- **Confirm the Dataverse first-party principal for GCC High (verify for GCC High).** Do not blindly grant the Commercial app id `00000007-0000-0000-c000-000000000000`; verify the actual principal in Entra ID → Enterprise applications and document its object id as an engagement artifact.

Secret hygiene

- **Set an expiration on every secret** and rotate on a schedule. `PPKVG-Report.ps1` flags secrets with no expiry.
- **Rotate in Key Vault, not in Power Platform.** The environment variable references the secret by location, so a new version flows through without touching Power Platform. This is the core payoff of the pattern.
- **Standardize secret names** (`Secret – <SystemName> – <EnvName>`) so references, audits, and rotation are predictable.
- **Consumers only:** environment-variable secrets work with **Power Automate flows, custom connectors, and Copilot Studio agents** — not general canvas-app formulas or the Web API. Design around that supported surface; don't try to read the value elsewhere.

Flow / consumer safety

- **Always enable Secure Inputs and Secure Outputs** on the Dataverse `RetrieveEnvironmentVariableSecretValue` step and on any step that uses the value, so secrets never appear in run history.
- **Never log or output the secret value.** Not in flow compose steps, not in traces, not in the scripts. The provided scripts read metadata only by design — keep any custom automation to the same rule.

Network posture

- **Prefer VNet + Private Link** for production so the vault has no public exposure; the private endpoint sits on the Power-Platform-delegated subnet.
- **If you use the Key Vault firewall, allow the Power Platform US Gov IP ranges explicitly** — Power Platform is **not** included in "Allow trusted Microsoft services." Use the **US Gov** IP set (`AzureCloud.usgovvirginia` / `AzureCloud.usgovtexas`), refreshed periodically.

Operational discipline

- **Run the read-only scripts before any change** and the validation script after — the WhatIf-by-default remediation script should always be previewed first.
- **Keep a per-environment governance report** (`PPKVG-Report.ps1`) as recurring evidence for auditors: vaults, permission models, secret names/expiry/tags, and Secrets User assignments — all without a single secret value.
- **Change control:** treat provider registration, RBAC changes, and network lockdown as change-managed actions (global requirement G5) with a rollback (G6): capture the prior IAM/network state before applying.
- **Watch for parity drift.** GCC High capabilities can lag Commercial; re-check the Business Applications US Government availability summary before relying on any newer feature, and flag anything unconfirmed as **"verify for GCC High."**

Deliverables Summary — Standardize Power Platform Secrets in Azure Key Vault (Government / GCC High)

Everything included in this solution and where to find it.

Documents (`source/`)

File	What it covers
`executive-summary.md`	One-page business case + the three biggest Commercial-vs-GCC-High differences.
`01-solution-overview.md`	What the solution does, the deployment model, and the **Commercial vs GCC High — what's different** section.

File	What it covers
`02-requirements.md`	Prerequisites R1–R8 with Verify / Implement-if-missing, layered on the global Azure requirements.
`03-implementation.md`	Ordered click-to-implement steps using the GCC High portals, with parity caveats inline and phase gates.
`05-testing.md`	Pass/fail tests mapped to each phase gate; evidence to capture.
`06-troubleshooting.md`	GCC-High-specific symptoms, causes, and fixes.
`07-best-practices.md`	Durable operating practices (vault design, least privilege without PIM, hygiene, network).
`gcc-high-parity-notes.md`	**Cited** Commercial-vs-GCC-High reference (endpoints, identity, feature mechanics, Azure Gov management gaps).
`08-deliverables-summary.md`	This index.
`global-requirements-azure-solutions.md`	Baseline Azure platform requirements (G1–G6) shared by all Azure solutions.

Scripts (`scripts/`)

All scripts connect to the sovereign cloud (`-Environment AzureUSGovernment`), are **read-only for secrets**, and **never read secret values** (metadata only). Remediation and scope-targeting default to **preview (WhatIf)** and require `-Apply` to change anything.

Script	Type	What it does
`PPKVG-Discovery.ps1`	Read-only	Inventories vaults, permission models, secret **names/metadata**, and Key Vault Secrets User assignments; checks the `Microsoft.PowerPlatform` provider. Outputs CSV.
`PPKVG-Validation.ps1`	Read-only	Pass/fail check of every prerequisite control for a target vault/secret (provider, RBAC model, Secrets User, Dataverse principal, network, secret present).
`PPKVG-Report.ps1`	Read-only	Governance HTML + CSV report with hygiene findings (no-expiry secrets, access-policy vaults, missing Secrets User).

Script	Type	What it does
`PPKVG-Remediation.ps1`	Change (WhatIf default)	Registers the provider, switches a vault to Azure RBAC, and grants Key Vault Secrets User to the Dataverse principal and makers group. Flags the Dataverse principal "verify for GCC High."
`PPKVG-Set-ScopeTargeting.ps1`	Change (WhatIf default)	Sets the Copilot Studio `AllowedBots` / `AllowedEnvironments` **tag** on a secret (metadata only — value untouched).

Running the scripts

```
# Modules (once): Install-Module Az -Scope CurrentUser
# All scripts sign in to Azure Government on first run.

powershell -ExecutionPolicy Bypass -File .\PPKVG-Discovery.ps1
powershell -ExecutionPolicy Bypass -File .\PPKVG-Validation.ps1 -VaultName 'your-vault-name' -SecretName '
powershell -ExecutionPolicy Bypass -File .\PPKVG-Report.ps1
powershell -ExecutionPolicy Bypass -File .\PPKVG-Remediation.ps1            # preview
powershell -ExecutionPolicy Bypass -File .\PPKVG-Remediation.ps1 -Apply     # apply
powershell -ExecutionPolicy Bypass -File .\PPKVG-Set-ScopeTargeting.ps1 -AllowedEnvironments 'your-environ
```

Packaged outputs

Artifact	Contents
`PPKVG-Solution.pdf`	The compiled document set (this `source/` folder).
`PPKVG-Scripts.zip`	The five PowerShell scripts.

Suggested engagement flow

1. Read `executive-summary.md` and the **Commercial vs GCC High** section of `01-solution-overview.md`.
2. Work `02-requirements.md`; run `PPKVG-Discovery.ps1` (Assess).
3. Follow `03-implementation.md` phase by phase; validate each gate with `05-testing.md` and the scripts.
4. Adopt `07-best-practices.md`; keep `06-troubleshooting.md` and `gcc-high-parity-notes.md` on hand.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PPKVG-Discovery.ps1

Purpose: Inventories the current state needed to standardize Power Platform secrets in Azure Key Vault on a US Government (GCC High) tenant. Lists key vaults, their permission model, secret NAMES and metadata (never values), and Key Vault Secrets User role assignments — so you can see what exists before you change anything. Set the placeholders below (for example your-subscription-id, your-vault-name). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PPKVG-Discovery.ps1 Modules: Az.Accounts, Az.KeyVault, Az.Resources (Install-Module Az -Scope CurrentUser). Connects to Azure Government: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS A console table plus a CSV of vaults, secret metadata, and role assignments. PASS: at least one vault is found and the Microsoft.PowerPlatform provider is Registered. FAIL/REVIEW: no vaults, provider NotRegistered, or vaults still on the vault-access-policy model (RBAC recommended) are flagged in the output. .NOTES White-label — the repeatable way. Read-only; never reads secret values. US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\PPKVG-Discovery.ps1 Modules: Az.Accounts, Az.KeyVault, Az.Resources (Install-Module Az -Scope CurrentUser). Connects to Azure Government: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS A console table plus a CSV of vaults, secret metadata, and role assignments. PASS: at least one vault is found and the Microsoft.PowerPlatform provider is Registered. FAIL/REVIEW: no vaults, provider NotRegistered, or vaults still on the vault-access-policy model (RBAC recommended) are flagged in the output. .NOTES White-label — the repeatable way. Read-only; never reads secret values. US Gov endpoints. #>

PPKVG-Remediation.ps1

Purpose: Applies the standard configuration for Power Platform environment-variable secrets on a US Government (GCC High) tenant: registers the Microsoft.PowerPlatform provider, switches the target vault to the Azure RBAC permission model, and grants Key Vault Secrets User to the Dataverse first-party principal and a named admin group. Runs in preview (-WhatIf) mode by default; pass -Apply to make changes. It NEVER creates, reads, or writes secret values. Set placeholders such as your-subscription-id, your-vault-name, and your-admin-group-object-id. HOW TO RUN Preview : powershell -ExecutionPolicy Bypass -File .\PPKVG-Remediation.ps1 Apply : powershell -ExecutionPolicy Bypass -File .\PPKVG-Remediation.ps1 -Apply Modules: Az.Accounts, Az.KeyVault, Az.Resources. Connects to Azure Government: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS Preview: prints each action it WOULD take, changes nothing. Apply: performs the actions and prints PASS/updated per step. Re-run PPKVG-Validation.ps1 afterward to confirm all controls are green. .NOTES White-label — the repeatable way. Read-only for secrets; never reads secret values. US Gov endpoints. #>

How to run: Preview : powershell -ExecutionPolicy Bypass -File .\PPKVG-Remediation.ps1 Apply : powershell -ExecutionPolicy Bypass -File .\PPKVG-Remediation.ps1 -Apply Modules: Az.Accounts, Az.KeyVault, Az.Resources. Connects to Azure Government: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS Preview: prints each action it WOULD take, changes nothing. Apply: performs the actions and prints PASS/updated per step. Re-run PPKVG-Validation.ps1 afterward to confirm all controls are green. .NOTES White-label — the repeatable way. Read-only for secrets; never reads secret values. US Gov endpoints. #>

PPKVG-Report.ps1

Purpose: Produces a shareable governance report for standardizing Power Platform secrets in Azure Key Vault on a US Government (GCC High) tenant. Summarizes vaults, permission model, secret metadata (names, expiry, tags — never values), Key Vault Secrets User assignments, and flags hygiene issues (secrets with no expiry, vaults on access-policy model, missing tags). Set the placeholders below such as your-subscription-id. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PPKVG-Report.ps1 Modules: Az.Accounts, Az.KeyVault, Az.Resources. Connects to Azure Government: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS Writes a timestamped PPKVG-Report .html and .csv to the output folder and prints a summary. PASS: report generated; review the Findings section for any REVIEW/ACTION rows. FAIL: connection or module missing — the error names what to install/fix. .NOTES White-label — the repeatable way. Read-only; never reads secret values. US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\PPKVG-Report.ps1 Modules: Az.Accounts, Az.KeyVault, Az.Resources. Connects to Azure Government: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS Writes a timestamped PPKVG-Report .html and .csv to the output folder and prints a summary. PASS: report generated; review the Findings section for any REVIEW/ACTION rows. FAIL: connection or module missing — the error names what to install/fix. .NOTES White-label — the repeatable way. Read-only; never reads secret values. US Gov endpoints. #>

PPKVG-Set-ScopeTargeting.ps1

Purpose: Builds and applies the metadata TAGS that scope which Copilot Studio agents (or which whole environments) may consume a Power Platform environment-variable secret on a US Government (GCC High) tenant. It sets the 'AllowedBots' or 'AllowedEnvironments' tag on the named secret's current version. It ONLY writes tag metadata — it never creates, reads, or changes the secret value. Runs in preview (-WhatIf) mode by default; pass -Apply to write the tag. Set placeholders such as your-vault-name, your-secret-name, and the environment/agent ids. HOW TO RUN Preview : powershell -ExecutionPolicy Bypass -File .\PPKVG-Set-ScopeTargeting.ps1 -AllowedEnvironments 'your-environment-id' Apply : powershell -ExecutionPolicy Bypass -File .\PPKVG-Set-ScopeTargeting.ps1 -AllowedEnvironments 'your-environment-id' -Apply Modules: Az.Accounts, Az.KeyVault. Connect: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS Preview: prints the exact tag name/value it WOULD set on the secret. Nothing changes. Apply: sets the tag and prints the resulting (metadata-only) tag set. PASS: tag is present on the secret; Copilot Studio agents in scope can now resolve it. .NOTES White-label — the repeatable way. Writes tag metadata only; never reads secret values. US Gov endpoints. #>

How to run: Preview : powershell -ExecutionPolicy Bypass -File .\PPKVG-Set-ScopeTargeting.ps1 -AllowedEnvironments 'your-environment-id' Apply : powershell -ExecutionPolicy Bypass -File .\PPKVG-Set-ScopeTargeting.ps1 -AllowedEnvironments 'your-environment-id' -Apply Modules: Az.Accounts, Az.KeyVault. Connect: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS Preview: prints the exact tag name/value it WOULD set on the secret. Nothing changes. Apply: sets the tag and prints the resulting (metadata-only) tag set. PASS: tag is present on the secret; Copilot Studio agents in scope can now resolve it. .NOTES White-label — the repeatable way. Writes tag metadata only; never reads secret values. US Gov endpoints. #>

PPKVG-Validation.ps1

Purpose: Validates the prerequisites for Power Platform environment-variable secrets on a US Government (GCC High) tenant: Microsoft.PowerPlatform provider registered, the target vault using the

RBAC permission model, the Dataverse first-party principal granted Key Vault Secrets User, network access reachable for Power Platform, and the named secret present (by name only). Set the placeholders below such as your-subscription-id and your-vault-name. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PPKVG-Validation.ps1 Modules: Az.Accounts, Az.KeyVault, Az.Resources. Connects to Azure Government: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS A checklist with PASS / FAIL / REVIEW per control and an overall result. PASS: every required control is green. FAIL: one or more required controls red — the line names the fix (see PPKVG-Remediation.ps1 / 03-implementation.md). .NOTES White-label — the repeatable way. Read-only; never reads secret values. US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\PPKVG-Validation.ps1 Modules: Az.Accounts, Az.KeyVault, Az.Resources. Connects to Azure Government: Connect-AzAccount -Environment AzureUSGovernment. EXPECTED RESULTS A checklist with PASS / FAIL / REVIEW per control and an overall result. PASS: every required control is green. FAIL: one or more required controls red — the line names the fix (see PPKVG-Remediation.ps1 / 03-implementation.md). .NOTES White-label — the repeatable way. Read-only; never reads secret values. US Gov endpoints. #>